

1. **Explain** the purpose of Big O Notation and **describe** how it helps us understand the efficiency of algorithms.

Big O Notation is a mathematical concept used in computer science to describe the **efficiency** of algorithms, particularly in terms of **time complexity** (how fast they run) and **space complexity** (how much memory they use).

Purpose of Big O Notation

1. **Measures Algorithm Efficiency:** It provides a way to express how the running time or memory usage of an algorithm grows as the size of the input (usually denoted as n) increases.
 - Example: Searching in a list of 10 elements vs. 1 million elements.
2. **Abstracts Away Machine Details:** Instead of focusing on exact seconds or memory units (which depend on hardware and programming language), Big O focuses on the **growth rate** as input size increases.
3. **Compares Algorithms Fairly:** It allows us to compare two or more algorithms regardless of the computer they run on. For instance, an algorithm that runs in $O(n \log n)$ will eventually outperform an $O(n^2)$ algorithm for sufficiently large inputs, even if the latter is faster on small inputs.

How It Helps Us Understand Efficiency

- **Scalability:** Big O tells us how an algorithm scales with larger input sizes.
 - Example:
 - Linear Search $\rightarrow O(n)$
 - Binary Search $\rightarrow O(\log n)$Binary search will be far more efficient for large datasets.
- **Identifies Bottlenecks:** By analyzing Big O, we can spot where an algorithm may become impractical (e.g., $O(n!)$ factorial time is infeasible for even moderately large inputs).
- **Guides Algorithm Choice:** In real-world applications, we often choose algorithms based on their Big O complexity to balance performance and resource use.

Common Big O Complexities

- $O(1) \rightarrow$ Constant time (best case, independent of input size)
 - $O(\log n) \rightarrow$ Logarithmic time (binary search)
 - $O(n) \rightarrow$ Linear time (simple loops, linear search)
 - $O(n \log n) \rightarrow$ Log-linear time (efficient sorting like mergesort, quicksort)
 - $O(n^2) \rightarrow$ Quadratic time (nested loops, bubble sort)
 - $O(2^n) \rightarrow$ Exponential time (recursive algorithms without optimization, like naive Fibonacci)
 - $O(n!) \rightarrow$ Factorial time (brute-force permutations, traveling salesman problem)
2. **Compare** different Big O complexities (e.g., $O(1)$, $O(n)$, $O(\log n)$, $O(n^2)$) using algorithm examples, and **justify** why each has its specific time complexity.

✔ Comparison Table		
Big O	Example Algorithm	Justification
$O(1)$	Array index access	Constant operation, independent of input
$O(\log n)$	Binary search	Divide input by 2 each step
$O(n)$	Linear search	One check per element
$O(n \log n)$	Merge Sort, Quick Sort	$\log n$ levels $\times$ n merging
$O(n^2)$	Bubble Sort, Insertion Sort	Nested loops over n elements
$O(2^n)$	Recursive Fibonacci	Branching recursion doubles calls
$O(n!)$	Permutations	All possible arrangements generated

3. **Analyze** a simple Python function that searches a list and **determine** its worst-case time complexity using Big O Notation, with reasoning.

```
def search_list(arr, target):

    for i in range(len(arr)):

        if arr[i] == target:

            return i

    return -1
```

Step 1: What does it do?

- The function loops through the list `arr`.
- For each element, it checks if it matches the target.
- If found, it returns the index; otherwise, it returns -1.

Step 2: Count the operations

- **Loop:** `for i in range(len(arr))` → iterates through all n elements (where n = length of list).
- **Comparison inside loop:** `arr[i] == target` → runs once per element.
- **Return:** either when found (could be early) or after the last element.

Step 3: Best Case vs Worst Case

- **Best Case ($O(1)$):** If the target is the **first element**, only 1 comparison is made.
- **Worst Case ($O(n)$):**
 - If the target is the **last element**, the function compares all n elements.
 - If the target is **not present**, it still checks all n elements before returning -1.

Step 4: Big O Analysis

- In **worst case**, the number of operations grows **proportionally to n** .
 - Thus, **time complexity** = **$O(n)$** .
4. **Differentiate** between time complexity and space complexity in algorithms and **explain** why both are important for evaluating performance.

1. Time Complexity

- **Definition:**
Time complexity measures how the **execution time** of an algorithm grows as the input size (n) increases.
- **Focus:** Speed (how many operations are required).
- **Measured Using:** Big O notation ($O(1)$, $O(n)$, $O(n^2)$, etc.).
- **Example:**
 - **Linear Search:** $O(n)$ → time grows linearly with input size.
 - **Binary Search:** $O(\log n)$ → time grows much slower.

2. Space Complexity

- **Definition:**
Space complexity measures how much **memory (RAM)** an algorithm needs to run as input size (n) increases.
- **Focus:** Memory (extra storage like arrays, hash tables, recursion stack, etc.).
- **Measured Using:** Big O notation ($O(1)$, $O(n)$, etc.).
- **Example:**
 - **Iterative Fibonacci:** $O(1)$ extra space (only variables).
 - **Recursive Fibonacci:** $O(n)$ space (due to recursive call stack).
 - **Merge Sort:** $O(n)$ extra memory for merging.

5. **Describe** what a string is in Python and **explain** how it is represented internally.

- A **string** in Python is a **sequence of characters** enclosed in single quotes ('...'), double quotes ("..."), or triple quotes ('''...''' or """"...""").

- Strings are used to represent text data like names, sentences, or symbols.

- **Examples:**

```
s1 = "Hello"
```

```
s2 = 'Python'
```

```
s3 = """This is a multi-line string"""
```

Strings in Python are **immutable**, meaning once created, they cannot be changed. Any modification creates a new string object.

2. Internal Representation of Strings

Internally, Python handles strings in a way that supports **Unicode**, so they can represent characters from virtually any language (English, Hindi, Chinese, emojis, etc.).

- **Unicode Support (UTF-8/UTF-16/UTF-32):**
 - Python 3 strings are stored as sequences of **Unicode code points**.
 - Each character has a unique code (e.g., "A" = U+0041, "😊" = U+1F60A).
 - Internally, CPython (the most common Python implementation) uses a flexible storage strategy called **PEP 393 (Flexible String Representation)**.
6. **Demonstrate** how to perform common string operations in Python—such as concatenation, slicing, and substring searching—using code examples.
 7. **Explain** the conceptual differences between arrays and Python lists, including how they are used in programming.

Aspect	Array	Python List
Definition	Collection of elements (usually same type) stored in contiguous memory.	High-level dynamic data structure that can hold elements of different types.
Data Type	Homogeneous (all elements must be of the same type).	Heterogeneous (can store different types in one list).
Size	Fixed size (in many languages).	Dynamic, can grow or shrink automatically.
Memory Storage	Contiguous memory locations → more efficient.	Dynamic allocation with extra overhead for flexibility.
Performance	Faster, more memory-efficient, especially for numerical data.	Slightly slower, uses more memory due to flexibility.
Resizing	Costly (requires new memory allocation).	Automatic and handled internally by Python.
Ease of Use	Requires explicit management (especially in low-level languages).	Very easy to use with built-in methods (<code>append</code> , <code>insert</code> , <code>remove</code> , etc.).
Use Cases	Performance-critical apps, numerical/scientific computing.	General-purpose programming, storing and manipulating collections of data.
Python Example	<code>array.array('i', [1, 2, 3])</code> → only integers.	<code>[1, "apple", 3.5, True]</code> → mixed data allowed.

8. **Show** how to create a list in Python, and **illustrate** how to add and remove elements with examples.

Creating a List in Python

```
my_list = []
```

```
fruits = ["apple", "banana", "cherry"]
```

```
numbers = [10, 20, 30, 40]
```

```
mixed = [1, "hello", 3.14, True]
```

Adding Elements to a List:

1. Append()
2. Insert()
3. Extend()

Removing Elements from a List:

1. Pop()
2. Remove()
3. Del statement
4. Clear()

9. **Describe** the concept of an Abstract Data Type (ADT) and **explain** the key characteristics that distinguish ADTs from data structures.

An **Abstract Data Type (ADT)** is a **theoretical model** for data organization that defines **what operations can be performed** on the data, but **not how** those operations are implemented.

It focuses on the **behavior (interface)** rather than the actual implementation (details).

Example: **Stack ADT** defines operations like `push()`, `pop()`, and `peek()` without specifying whether it's implemented using an array or a linked list.

Key Characteristics of ADTs

1. **Abstraction**
 - ADTs hide the internal implementation details from the user.
 - The user only knows the operations (functions/methods) available, not how they work internally.
2. **Well-defined Operations**
 - ADTs specify a set of valid operations.
 - Example: For a **Queue ADT**, operations are `enqueue()` and `dequeue()`.
3. **Encapsulation**
 - The data and operations are bundled together, restricting direct access to internal representation.
4. **Implementation Independence**
 - The same ADT can be implemented using different data structures.
 - Example: A **Queue ADT** can be implemented using arrays, linked lists, or even stacks.

10. **Explain** the importance of Abstract Data Types in software development, especially when designing systems in AI or machine learning.

Importance of Abstract Data Types (ADTs) in Software Development

Abstract Data Types (ADTs) are fundamental because they provide a **conceptual framework** for organizing and managing data efficiently while hiding implementation details. Their importance comes from several angles:

1. Abstraction and Modularity

- ADTs allow developers to **focus on what operations can be performed**, not on *how* they are carried out.
- This makes code **cleaner, modular, and easier to maintain**.
- Example: A **Queue ADT** can be used in a scheduling system without worrying whether it's implemented with an array or linked list.

2. Reusability and Flexibility

- Once an ADT is defined, it can be **reused across projects** with different underlying implementations.
- Example: A **Graph ADT** can be implemented using adjacency lists or adjacency matrices depending on performance needs.

3. Improved Maintainability

- If the internal representation changes (e.g., switching from an array-based stack to a linked-list-based stack), the **interface remains the same**.
- This allows developers to **improve performance or memory usage** without breaking existing code.

4. Efficiency in Problem Solving

- Choosing the right ADT ensures **efficient data manipulation**, which is critical for performance.
- Example: A **Priority Queue ADT** helps optimize scheduling tasks in operating systems or AI algorithms.

Importance of ADTs in AI and Machine Learning

AI and ML systems deal with **large, complex, and dynamic datasets**, making ADTs especially valuable.

1. Managing Large Datasets Efficiently

- AI/ML models often process **huge volumes of data** (images, text, time series).
- ADTs like **Hash Tables (for fast lookups)**, **Heaps (for priority operations)**, and **Graphs (for relationships/networks)** are essential.

2. Enabling Algorithmic Optimization

- Many ML/AI algorithms rely on efficient data handling.
- Example:
 - **Graphs ADT** → used in neural networks, recommendation systems, and social network analysis.

- **Matrix ADT** → used in linear algebra for training models.

3. Flexibility in Experimentation

- Researchers can **swap out implementations** (e.g., dense vs sparse matrix) without rewriting the algorithm.
- This allows **rapid prototyping** and **faster experimentation** in AI/ML development.

4. Encapsulation for Complex Systems

- AI/ML pipelines involve multiple components (data preprocessing, training, inference).
- By using ADTs, each component can work with **defined interfaces**, improving **collaboration and scalability**.

5. Real-World AI/ML Examples

- **Priority Queue ADT** → used in A* search algorithms for pathfinding in robotics.
- **Graph ADT** → used in knowledge graphs, social networks, and recommendation engines.
- **Matrix/Vector ADTs** → backbone of deep learning (matrix multiplications, tensors).
- **Hash Tables ADT** → for quick feature lookups in Natural Language Processing (NLP).

11. **Identify and describe** common ADTs such as List ADT, Stack ADT, and Queue ADT, including a brief example of each.

1. List ADT

Definition

- A **List ADT** is an ordered collection of elements where each element has a position (index).
- Allows **insertion, deletion, traversal, and searching**.
- Can be implemented using **arrays** or **linked lists**.

Key Operations

- `insert(position, element)` → Insert at specific index
- `delete(position)` → Remove element at index
- `get(position)` → Retrieve element by index
- `traverse()` → Visit all elements

Using Python's built-in list to represent List ADT

```
my_list = [10, 20, 30]
```

```
my_list.insert(1, 15) # Insert 15 at index 1
```

```
print(my_list)      # [10, 15, 20, 30]
```

2. Stack ADT

Definition

- A **Stack ADT** is a collection of elements that follows the **LIFO (Last In, First Out)** principle.
- Think of it like a **stack of plates** – the last plate placed is the first one removed.

Key Operations

- `push(element)` → Add element to the top
- `pop()` → Remove and return the top element
- `peek()` → View the top element without removing
- `isEmpty()` → Check if stack is empty

Example in Python

```
stack = []

stack.append(10) # Push

stack.append(20) # Push

print(stack.pop()) # Pop → 20

print(stack)      # [10]
```

3. Queue ADT

Definition

- A **Queue ADT** is a collection of elements that follows the **FIFO (First In, First Out)** principle.
- Like a **line at a ticket counter** – the first person in is the first person served.

Key Operations

- `enqueue(element)` → Add element at the rear
- `dequeue()` → Remove and return element from the front
- `peek()` → View the front element without removing
- `isEmpty()` → Check if queue is empty

Example in Python

```
from collections import deque

queue = deque()

queue.append(10) # Enqueue
```



```
queue.append(20) # Enqueue

print(queue.popleft()) # Dequeue → 10

print(queue)      # deque([20])
```

12. **Explain** how conditional statements (if, elif, else) control the flow of a Python program, using examples.

Conditional Statements in Python

Conditional statements allow a program to **make decisions** and **execute different code paths** depending on whether a condition is True or False. In Python, these are handled with if, elif, and else

1. if Statement

- Tests a condition.
- Executes a block of code **only if** the condition is True.

```
x = 10

if x > 5:

    print("x is greater than 5")
```

Output: x is greater than 5

2. if-else Statement

- Provides an **alternative block** if the condition is False.

```
x = 3

if x > 5:

    print("x is greater than 5")
```

```
else:

    print("x is 5 or less")
```

Output: x is 5 or less

3. if-elif-else Chain

- Checks **multiple conditions** in sequence.
- The first condition that evaluates to True executes, and the rest are skipped.
- If none match, the else block runs.

```
score = 85

if score >= 90:

    print("Grade: A")

elif score >= 75:

    print("Grade: B")

elif score >= 60:

    print("Grade: C")

else:

    print("Grade: F")
```

Output: Grade: B

4. Nested if Statements

- You can put an if inside another if.

```
x = 12

if x > 5:

    if x % 2 == 0:

        print("x is greater than 5 and even")
```

Output: x is greater than 5 and even

13. **Write** a Python program that takes an integer input and **uses** conditional statements to check if the number is positive, negative, or zero.

```
# Program to check if a number is positive, negative, or zero

# Take integer input from user

num = int(input("Enter an integer: "))

# Conditional checks

if num > 0:

    print("The number is Positive")

elif num < 0:
```

```
print("The number is Negative")
```

else:

```
print("The number is Zero")
```

14. **Explain** the purpose of loops in programming and **differentiate** when to use a `for` loop versus a `while` loop in Python.

Purpose of Loops in Programming

- A **loop** allows you to **repeat a block of code multiple times** without writing it again.
- Instead of manually repeating instructions, loops **automate repetition** until a condition is met.

This helps in:

- Reducing code duplication
- Making programs shorter and cleaner
- Handling tasks with unknown or large repetitions

Types of Loops in Python

Python mainly has two loops: **for loop** and **while loop**.

1. for Loop

- Used when the **number of iterations is known** or when iterating over a sequence (list, string, range, tuple, etc.).
- Best for **definite iteration**.

Example

```
# Print numbers 1 to 5 using for loop
```

```
for i in range(1, 6):
```

```
    print(i)
```

Output:

1

2

3

4

5

2. while Loop

- Used when the **number of iterations is not known in advance**.
- Runs as long as the condition is **True**.
- Best for **indefinite iteration**.

Example

```
# Print numbers 1 to 5 using while loop
```

```
i = 1
```

```
while i <= 5:
```

```
    print(i)
```

```
    i += 1
```

Output:

1

2

3

4

5

15. **Write** a Python program using a `for` loop to print the numbers 1 through 5.

```
# Program to print numbers from 1 to 5 using a for loop
```

```
for i in range(1, 6):
```

```
    print(i)
```

16. **Write** a Python program using a `while` loop to calculate and print the sum of numbers from 1 to 10.

```
# Program to calculate sum of numbers from 1 to 10 using while loop
```

```
# Initialize variables
```

```
i = 1    # Counter
```

```
total = 0 # To store sum
```

```
# Loop from 1 to 10

while i <= 10:

    total += i # Add current number to total

    i += 1     # Increment counter

# Print the result

print("The sum of numbers from 1 to 10 is:", total)
```

17. **Explain** how variables work in Python and **demonstrate** how to declare and assign values using code examples.

What is a Variable?

- A **variable** is a **named storage** in memory used to store data that a program can manipulate.
- Variables allow programs to **store, retrieve, and update values** dynamically.
- In Python, variables are **created when you first assign a value**. You don't need to declare a type explicitly.

Key Points About Variables in Python

1. **Dynamic Typing**
 - Python variables **don't need a type declaration**; the type is determined automatically.
2. `x = 5` # integer
3. `y = 3.14` # float
4. `name = "Alice"` # string
5. **Assignment**
 - Use the `=` operator to assign a value to a variable.
6. `age = 25`
7. **Variable Naming Rules**
 - Must start with a **letter** or **underscore** (`_`)
 - Can contain **letters, digits, or underscores**
 - Cannot be a **Python keyword** (e.g., `if`, `for`, `while`)
8. **Reassignment**
 - Variables can be **updated** anytime.
9. `score = 50`
10. `score = 75` # reassigned

Examples of Declaring and Assigning Variables

Numbers

```
a = 10        # integer
```

```
b = 3.5       # float
```

```
# String

name = "Gauri"

# Boolean

is_active = True

# Multiple assignments in one line

x, y, z = 1, 2, 3

# Updating a variable

x = x + 5 # x now becomes 6
```

Printing Variables

```
print("a =", a)

print("b =", b)

print("Name:", name)

print("Is Active?", is_active)

print("x =", x, "y =", y, "z =", z)
```

Output:

a = 10

b = 3.5

Name: Gauri

Is Active? True

x = 6 y = 2 z = 3

18. **Identify** common data types in Python (integer, float, string, boolean) and **demonstrate** each with a relevant example.

1. Integer (int)

- Used for **whole numbers** (positive, negative, or zero).

```
# Integer example
```

```
age = 25
```

```
year = 2025

print("Age:", age)

print("Year:", year)
```

✅ Output:

```
Age: 25

Year: 2025
```

2. Float (float)

- Used for **numbers with decimals**.

```
# Float example

height = 5.9

weight = 70.5

print("Height:", height)

print("Weight:", weight)
```

Output:

```
Height: 5.9

Weight: 70.5
```

3. String (str)

- Used for **text**. Strings are enclosed in **quotes** (' ' or " ").

```
# String example

name = "Alice"

greeting = 'Hello, world!'

print("Name:", name)

print(greeting)
```

Output:

```
Name: Alice
```

Hello, world!

4. Boolean (bool)

- Represents **logical values**: True or False.

Boolean example

```
is_active = True
```

```
is_logged_in = False
```

```
print("Active?", is_active)
```

```
print("Logged In?", is_logged_in)
```

Output:

Active? True

Logged In? False

Q1. Explain the insertion operation in linked list. How nodes are inserted after a specified node.

Linked List Overview

A **linked list** is a linear data structure where each element, called a **node**, contains:

1. **Data** – the value stored in the node.
2. **Pointer (or Next)** – reference to the next node in the list.

Example node structure in Python:

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

A linked list has a **head node** pointing to the first element.

Insertion in a Linked List

Insertion refers to **adding a new node** at a specific location in the list.

Types of Insertion

1. **At the beginning**
2. **At the end**
3. **After a specified node** (our focus)

Insertion After a Specified Node

To insert a node **after a given node**:

Steps

1. Create a **new node** with the data to be inserted.
2. Point the next of the new node to the next of the specified node.
3. Change the next of the specified node to point to the **new node**.

Diagram Representation

Suppose we have nodes: $A \rightarrow B \rightarrow C$

- We want to insert X **after B**:

Before insertion: $A \rightarrow B \rightarrow C$

Step 1: Create X

Step 2: $X.next = B.next$ (C)

Step 3: $B.next = X$

After insertion: $A \rightarrow B \rightarrow X \rightarrow C$

Python Code Example

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    # Function to insert after a given node
    def insert_after(self, prev_node, data):
        if prev_node is None:
            print("The given previous node must not be None.")
            return
        new_node = Node(data)
        new_node.next = prev_node.next
        prev_node.next = new_node
    # Function to print linked list
    def print_list(self):
        temp = self.head
        while temp:
            print(temp.data, end=" → ")
            temp = temp.next
        print("None")
# Example usage
l1 = LinkedList()
l1.head = Node("A")
second = Node("B")
third = Node("C")
# Linking nodes
l1.head.next = second
second.next = third
print("Original list:")
l1.print_list()
# Insert X after B
l1.insert_after(second, "X")
print("After inserting X after B:")
l1.print_list()
```

Output:

Original list:

$A \rightarrow B \rightarrow C \rightarrow \text{None}$

After inserting X after B:

$A \rightarrow B \rightarrow X \rightarrow C \rightarrow \text{None}$

Q2. Illustrate how stack is used in reversing the string

A **stack** is a data structure that follows **LIFO (Last In, First Out)**:

- The **last element pushed** onto the stack is the **first one popped**.

When reversing a string using a stack:

1. **Push** all characters of the string onto the stack.
2. **Pop** characters one by one from the stack and append them to a new string.
3. The result is the **reversed string**.

Step-by-Step Example

Original string: "HELLO"

1. **Push characters onto stack:**

Stack after each push

H

H, E

H, E, L

H, E, L, L

H, E, L, L, O

2. **Pop characters from stack and append:**

Pop	Reversed string
-----	-----------------

O	"O"
---	-----

L	"OL"
---	------

L	"OLL"
---	-------

E	"OLLE"
---	--------

H	"OLLEH"
---	---------

Result: "OLLEH" 

Python Implementation

```
def reverse_string(s):
    stack = []
    # Push all characters onto stack
    for char in s:
        stack.append(char)
    # Pop characters to get reversed string
    reversed_str = ""
    while stack:
        reversed_str += stack.pop()
    return reversed_str

# Example usage
string = "HELLO"
reversed_string = reverse_string(string)
```

```
print("Original String:", string)
print("Reversed String:", reversed_string)
```

Output:

Original String: HELLO
Reversed String: OLLEH

Q3.Design and explain a stack-based algorithm to check if a string is a palindrome.

- A **palindrome** is a string that reads the **same forward and backward**.
Example: "RADAR", "level"
- **Stack-based approach** leverages the **LIFO (Last In, First Out)** property of stacks:
 1. Push all characters of the string onto the stack.
 2. Pop characters one by one and **compare** with the original string.
 3. If all characters match in sequence, it's a palindrome.

Step-by-Step Algorithm

Input: String s

Output: True if s is a palindrome, else False

Algorithm

1. Create an empty stack `stack = []`.
2. **Push** each character of s onto the stack.
3. Initialize an empty string `reversed_str = ""`.
4. While the stack is not empty:
 - Pop a character from the stack.
 - Append it to `reversed_str`.
5. Compare `reversed_str` with s:
 - If equal → s is a palindrome.
 - Else → s is not a palindrome.

Python Implementation

```
def is_palindrome(s):
```

```
    stack = []
```

```
    # Push all characters onto the stack
```

```
    for char in s:
```

```
        stack.append(char)
```

```
    # Pop characters to build reversed string
```

```

reversed_str = ""
while stack:
    reversed_str += stack.pop()
# Check if reversed string equals original
if s == reversed_str:
    return True
else:
    return False
# Example usage
string1 = "RADAR"
string2 = "HELLO"
print(string1, "is palindrome?", is_palindrome(string1))
print(string2, "is palindrome?", is_palindrome(string2))

```

Output

RADAR is palindrome? True

HELLO is palindrome? False

Q4. Write an algorithm to perform add, delete and display operation using LIST ADT

List ADT Operations Algorithm

Assumptions:

- The list can store elements of any type.
- Operations:
 1. **Add** → Insert an element at a specified position or at the end.
 2. **Delete** → Remove an element by value or index.
 3. **Display** → Print all elements in the list.

Algorithm Steps

Initialize

1. Create an empty list: `my_list = []`

1. Add Operation

Input: Element E, optional position pos

Steps:

1. If pos is provided and valid ($0 \leq \text{pos} \leq \text{length of list}$):

- Insert E at position pos.
- 2. Else:
 - Append E at the end of the list.
- 3. Display success message.

2. Delete Operation

Input: Element E or index pos

Steps:

1. If deleting by value:
 - Search for E in the list.
 - If found → remove it.
 - Else → display "Element not found".
2. If deleting by position pos:
 - Check if pos is valid ($0 \leq \text{pos} < \text{length of list}$).
 - If valid → remove element at pos.
 - Else → display "Invalid position".
3. Display success message.

3. Display Operation

Steps:

1. If list is empty:
 - Print "List is empty".
2. Else:
 - Traverse the list and print all elements.

4. Repeat Operations

- Allow the user to choose operations repeatedly until they **exit**.

Q5. Discuss the various Operations performed on the Stack with suitable example.

Basic Stack Operations

Operation	Description	Example in Python
<code>push()</code>	Adds an element to the top of the stack.	<code>stack.append(10)</code>
<code>pop()</code>	Removes and returns the top element.	<code>stack.pop()</code>
<code>peek()/top()</code>	Returns the top element without removing it.	<code>stack[-1]</code>
<code>isEmpty()</code>	Checks whether the stack is empty.	<code>len(stack) == 0</code>
<code>size()</code>	Returns the number of elements in the stack.	<code>len(stack)</code>

Initialize an empty stack

```
stack = []
```

1. `push()` - Adding elements

```
stack.append(10)
```

```
stack.append(20)
```

```
stack.append(30)
```

```
print("Stack after pushes:", stack) # Output: [10, 20, 30].
```

2. `pop()` - Removing top element

```
top_element = stack.pop()
```

```
print("Popped element:", top_element) # Output: 30
```

```
print("Stack now:", stack) # Output: [10, 20]
```

3. `peek()` - Viewing top element without removing

```
print("Top element:", stack[-1]) # Output: 20
```

4. `isEmpty()` - Checking if stack is empty

```
print("Is stack empty?", len(stack) == 0) # Output: False
```

5. `size()` - Number of elements in stack

```
print("Size of stack:", len(stack)) # Output: 2
```

Q6. Explain the step-by-step process of evaluating a postfix expression using a stack.

Use the example:

"5 6 2 + * 12 4 / -"

Q7. Explain the step-by-step process of evaluating a prefix expression using a stack.

$+ - * 235 / ^ 234$

Q8. Write an algorithm for converting infix expression into postfix expression.

Infix to Postfix Conversion

Concept

- **Infix expression:** Operators are written **between operands** (e.g., $A + B$)
- **Postfix expression:** Operators are written **after operands** (e.g., $A B +$)
- **Why use stack:**
 - To **temporarily store operators** and ensure correct **operator precedence**.

Algorithm (Step-by-Step)

Input: Infix expression (e.g., $(A + B) * C$)

Output: Postfix expression (e.g., $A B + C *$)

Step 1: Initialize

1. Create an empty stack S for operators.
2. Create an empty list or string postfix for the output.

Step 2: Scan the infix expression from left to right

For each symbol ch in the infix expression:

1. **If ch is an operand** (letter or number):
 - Append ch to postfix.
2. **If ch is a left parenthesis $($:**
 - Push it onto the stack.
3. **If ch is a right parenthesis $)$:**
 - Pop operators from the stack and append to postfix **until a left parenthesis $($ is found**.
 - Pop and discard the left parenthesis $($.
4. **If ch is an operator $(+, -, *, /, ^)$:**
 - While the stack is **not empty** and the **top of the stack has higher or equal precedence** than ch :
 - Pop the operator from the stack and append it to postfix.
 - Push ch onto the stack.

Step 3: End of Expression

- After scanning the entire infix expression:

- Pop all remaining operators from the stack and append them to postfix.

Step 4: Return

- The postfix list or string is the **equivalent postfix expression**.
1. Use stack to convert infix expression into postfix expression.
 2. $A+B*(C^D-E)$
 3. Use stack to convert infix expression into prefix expression.
 4. $(C^D-E)^(F+G/H)$

Q10. Implement a stack using a Python list and write methods for push(), pop(), and get_min() (which returns the minimum element in O(1) time).

class MinStack:

def __init__(self):

self.stack = [] # Main stack

self.min_stack = [] # Auxiliary stack to track minimum

Push element onto stack

def push(self, value):

self.stack.append(value)

If min_stack is empty or new value <= current min, push it to min_stack

if not self.min_stack or value <= self.min_stack[-1]:

self.min_stack.append(value)

Pop element from stack

def pop(self):

if not self.stack:

print("Stack is empty!")

return None

value = self.stack.pop()

if value == self.min_stack[-1]:

self.min_stack.pop()

return value

Get minimum element in O(1) time

def get_min(self):

if not self.min_stack:


```

        print("Stack is empty!")

        return None

    return self.min_stack[-1]

# Peek top element
def peek(self):
    if not self.stack:
        print("Stack is empty!")
        return None
    return self.stack[-1]

# Example Usage
stack = MinStack()
stack.push(5)
stack.push(2)
stack.push(8)
stack.push(1)
print("Current Stack:", stack.stack)
print("Minimum Element:", stack.get_min()) # Output: 1
stack.pop()
print("After popping top element:")
print("Current Stack:", stack.stack)
print("Minimum Element:", stack.get_min()) # Output: 2
stack.pop()
print("After popping again:")
print("Current Stack:", stack.stack)
print("Minimum Element:", stack.get_min()) # Output: 2

```

Q11. Compare simple linear queues and circular queues in terms of memory efficiency and performance. In what type of real-time application would circular queues be more appropriate?

Real-Time Applications Where Circular Queues Are Appropriate

1. CPU Scheduling (Round Robin):

- Processes are scheduled in a circular manner; circular queue efficiently handles continuous rotation.

2. I/O Buffers:

- Keyboard, printer, or network buffers often use circular queues for continuous input/output handling.

3. Data Streaming / Real-Time Audio or Video Buffers:

- Circular queues efficiently manage streaming data with limited buffer space.

4. Traffic Control Systems:

- Managing requests or vehicles in a repetitive cycle without wasting memory.

Aspect	Linear Queue	Circular Queue 
Structure	Elements are stored in a linear sequence. Front moves forward as elements are dequeued.	Elements are stored in a circular array where the last position connects back to the first.
Memory Usage	Inefficient. Even if there is free space at the beginning, once the rear reaches the end, new elements cannot be added.	Efficient. Can reuse freed space at the beginning after elements are dequeued.
Overflow Condition	Occurs when <code>rear</code> reaches the end of the array, even if there are empty slots at the beginning.	Occurs only when the queue is truly full. Space is reused circularly, reducing wasted memory.
Implementation Complexity	Simple to implement.	Slightly more complex due to modulo operation for circular indexing.
Time Complexity (Enqueue/Dequeue)	$O(1)$ for both operations.	$O(1)$ for both operations.
Performance	Wasted memory can degrade performance if many enqueue/dequeue operations happen.	Better memory utilization; consistent performance in continuous operations.

Q12. Explain how a stack can be used to convert a decimal number to its binary equivalent.

Concept

- A **binary number** is composed of 0s and 1s.
- To convert a decimal number N to binary:
 1. **Divide N by 2**, record the **remainder**.
 2. Update $N = N // 2$.
 3. Repeat until N becomes 0.
- The **binary equivalent** is the sequence of remainders **read in reverse order**.

Using a **stack**, we can easily reverse the order of remainders because of the **LIFO (Last In, First Out)** property.

Step-by-Step Algorithm Using a Stack

Input: Decimal number N

Output: Binary equivalent

1. Initialize an empty stack.
2. While $N > 0$:
 - Compute remainder $r = N \% 2$.
 - Push r onto the stack.
 - Update $N = N // 2$.
3. Pop all elements from the stack and append them to a string or print → this is the binary number.

Example

Convert 13 to binary:

Step Decimal N $N \% 2$ (remainder) Push onto Stack

1	13	1	Stack: [1]
2	6	0	Stack: [1,0]
3	3	1	Stack: [1,0,1]
4	1	1	Stack: [1,0,1,1]
5	0	-	Stop

Pop elements from stack: 1 1 0 1 → Binary = 1101 

Python Implementation

```
def decimal_to_binary(n):  
    stack = []  
  
    # Step 1 & 2: Push remainders onto stack  
    while n > 0:  
        remainder = n % 2  
        stack.append(remainder)  
        n = n // 2  
  
    # Step 3: Pop from stack to get binary number  
    binary = ""  
    while stack:  
        binary += str(stack.pop())  
  
    return binary  
  
# Example usage
```

```
num = 13

binary_num = decimal_to_binary(num)

print(f"Binary of {num} is {binary_num}")
```

Output:

Binary of 13 is 1101

Explanation

1. The **stack stores remainders** in the order they are computed.
2. Popping from the stack **reverses the order**, giving the correct binary sequence.
3. Time complexity: **$O(\log N)$** since we divide N by 2 repeatedly.
4. Space complexity: **$O(\log N)$** for storing remainders in the stack.

Q13. Explain the difference between implementing a queue using an array vs. a linked list.

Aspect	Array Implementation	Linked List Implementation
Memory Allocation	Fixed size (static). Maximum queue size must be known in advance.	Dynamic memory allocation. Can grow or shrink as needed.
Memory Efficiency	May waste memory if the array is large but partially used; may overflow if full.	Efficient; only uses memory needed for current elements.
Insertion (Enqueue)	$O(1)$ if rear index is managed properly (circular array recommended); otherwise, $O(n)$ for shifting.	$O(1)$ if rear pointer is maintained.
Deletion (Dequeue)	$O(1)$ for circular array; $O(n)$ for simple array if shifting is required.	$O(1)$ if front pointer is maintained.
Implementation Complexity	Simple to implement. Need to handle index overflow (especially in linear array).	Slightly more complex due to node creation and pointers.
Overflow Condition	Occurs if array is full (even if there is space at beginning in linear array).	Never occurs unless system memory is exhausted.
Access by Index	$O(1)$ random access possible.	$O(n)$ access by index; must traverse nodes sequentially.
Use Cases	When maximum size is known in advance and memory is not a constraint.	When queue size is dynamic or memory utilization needs to be efficient.

Q14. Discuss task queue in Machine Learning workflow.

Task Queue?

- A **task queue** is a **data structure or system** that manages tasks to be executed **asynchronously**.

- It allows tasks to be **queued** for processing, rather than executed immediately.
- Typically used in systems where tasks are **time-consuming or resource-intensive**, allowing them to run in the **background**.

Key Components of a Task Queue System:

1. **Producer** → creates tasks and places them in the queue.
2. **Queue** → holds tasks until a worker is available.
3. **Worker** → consumes tasks from the queue and executes them.

Popular Tools: Celery, RabbitMQ, Kafka, Redis queues, etc.

2. Importance in Machine Learning Workflow

In ML workflows, there are often tasks that are **computationally heavy, time-consuming, or independent**, such as:

- Data preprocessing and cleaning.
- Feature extraction.
- Model training on large datasets.
- Hyperparameter tuning.
- Batch predictions or inference on large volumes of data.

Example ML Workflow Using Task Queue

Scenario: Batch prediction using a trained ML model on incoming data.

1. **Producer:**
 - Receives new user data and creates a prediction task.
 - Places task in the **task queue**.
2. **Queue:**
 - Stores tasks temporarily until workers are available.
3. **Worker(s):**
 - Pick tasks from the queue.
 - Perform preprocessing on input data.
 - Apply the trained ML model for prediction.
 - Store results in database or return to user.

Q15. Differentiate between array with linked list

Q16. Discuss the applications of linked list.

Applications of Linked List

A **linked list** is a dynamic data structure that stores elements (nodes) where each node points to the next (and sometimes previous) node. Its flexibility and efficient insertion/deletion make it suitable for many applications.

1. Dynamic Memory Allocation

- Linked lists allow **efficient memory usage** because memory is allocated as needed.
- Example: **Implementing stacks and queues** dynamically without knowing the size in advance.

2. Implementation of Stacks and Queues

- **Stack** (LIFO) → can be implemented using **singly linked lists** for dynamic push/pop operations.
- **Queue** (FIFO) → can be implemented using **singly or doubly linked lists** for dynamic enqueue/dequeue operations.

3. Undo/Redo Operations in Applications

- Applications like **text editors** use linked lists to store history:
 - Each action can be stored as a node.
 - **Undo** → move to previous node.
 - **Redo** → move to next node.

4. Dynamic Data Structures

- Linked lists are used as building blocks for more complex structures:
 - **Graphs** → adjacency lists use linked lists to store neighbors.
 - **Hash tables** → chaining for collision resolution uses linked lists.
 - **Sparse matrices** → only non-zero elements stored using linked lists.

5. Music or Video Playlists

- **Doubly linked lists** are ideal for playlists:
 - Forward → next song/video
 - Backward → previous song/video

6. Browser History / Navigation

- Browsers store history in a **doubly linked list**:
 - Back → previous page
 - Forward → next page

7. Polynomial Arithmetic

- Polynomials can be represented as linked lists:
 - Each node stores **coefficient** and **power**.

- Easy to add, subtract, and multiply polynomials using linked list operations.

8. Real-Time Applications

- Operating systems use linked lists for:
 - **Process scheduling**
 - **Memory management (free/allocated memory blocks)**
 - **Task queues** in asynchronous systems